arXiv:2411.14762v4 [cs.CV] 3 Apr 2025

Efficient Long Video Tokenization via Coordinate-based Patch Reconstruction

Huiwon Jang[1] Sihyun Yu[1] Jinwoo Shin[1] Pieter Abbeel[2] Younggyo Seo[2]
[1]KAIST [2]UC Berkeley

Abstract

Efficient tokenization of videos remains a challenge in training vision models that can process long videos. One promising direction is to develop a tokenizer that can encode long video clips, as it would enable the tokenizer to leverage the temporal coherence of videos better for tokenization. However, training existing tokenizers on long videos often incurs a huge training cost as they are trained to reconstruct all the frames at once. In this paper, we introduce CoordTok, a video tokenizer that learns a mapping from coordinate-based representations to the corresponding patches of input videos, inspired by recent advances in 3D generative models. In particular, CoordTok encodes a video into factorized triplane representations and reconstructs patches that correspond to randomly sampled (x, y, t) coordinates. This allows for training large tokenizer models directly on long videos without requiring excessive training resources. Our experiments show that CoordTok can drastically reduce the number of tokens for encoding long video clips. For instance, CoordTok can encode a 128-frame video with 128×128 resolution into 1280 tokens, while baselines need 6144 or 8192 tokens to achieve similar reconstruction quality. We further show that this efficient video tokenization enables memory-efficient training of a diffusion transformer that can generate 128 frames at once.

1. Introduction

Efficient tokenization of videos remains a challenge in developing vision models that can process long videos. While recent video tokenizers have achieved higher compression ratios [1, 2, 11, 54, 63, 64] compared to using image tokenizers for videos (*i.e.*, frame-wise compression) [45, 69], the vast scale of video data still requires us to design a more efficient video tokenizer.

One promising direction for efficient video tokenization is enabling video tokenizers to exploit the *temporal coherence* of videos. For instance, video codecs [28, 30, 34, 43] extensively utilize such coherence for video compression by extracting keyframes and encoding the difference between them. In fact, there have been several recent works based on a similar intuition that train a tokenizer to encode videos into factorized representations [21, 66, 67]. However, a key limitation is that existing tokenizers are typically trained to encode short video clips because of high training cost, but it is more likely that tokenizers can better exploit the temporal coherence when they are trained on longer videos. For instance, because tokenizers are trained to reconstruct all the frames at once, their training cost increases linearly with the length of videos (see Figure 1a). This makes it difficult to train tokenizers that can encode long videos and thus capture the temporal coherence of videos (see Figure 1b).

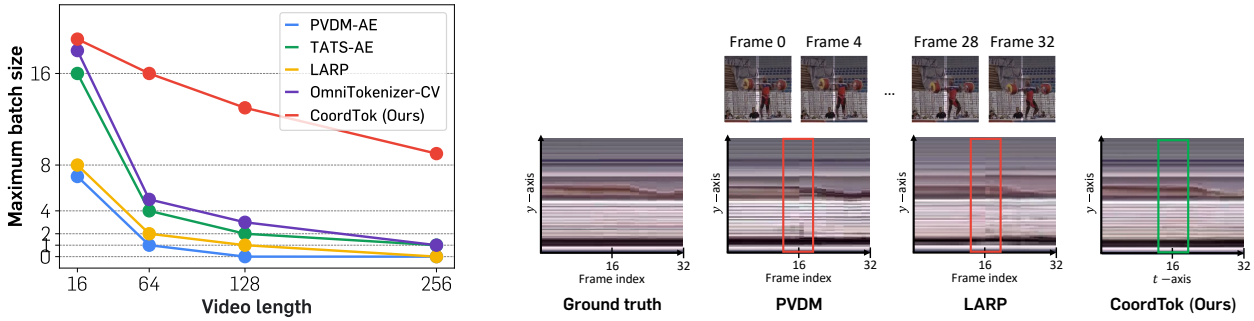
In this paper, we aim to design a video tokenizer that can be easily scaled up to encode long videos. To this end, we draw inspiration from recent works that have successfully trained large 3D generative models in a compute-efficient manner [18, 19, 24, 29]. Their key idea is to train a model that learns a mapping from randomly sampled (x, y, z) coordinates to RGB and density values instead of training with all the possible coordinates at once.

In particular, we ask: can we utilize a similar idea to design a scalable video tokenizer? Actually, there have been recent studies that formulate the video reconstruction as a problem of learning the mapping from (x, y, t) coordinates to RGB values [6, 22]. However, they rather focus on compressing each individual video instead of training a video tokenizer that can encode a diverse set of videos.

We introduce CoordTok: **Coord**inate-based patch reconstruction for long video **Tok**enization, a scalable video tokenizer that learns a mapping from coordinate-based representations to the corresponding patches of input videos. The key idea of CoordTok is to encode a video into factorized triplane representations [22, 66] and reconstruct patches that correspond to randomly sampled (x, y, t) coordinates (see Figure 2). This enables the training of large tokenizers directly on long videos without excessive memory and computational requirements (see Figure 1a).

To investigate whether training a video tokenizer on long video clips indeed leads to more efficient tokenization, we compare CoordTok with other baselines [11, 52, 63, 66] on the UCF-101 dataset [42]. Our experiments show that, by

Project website: huiwon-jang.github.io/coordtok
Correspondence to mail@younggyo.me.



(a) **Maximum batch-size when training video tokenizers** on 128×128 resolution videos with varying lengths, measured with a single NVIDIA 4090 24GB GPU.

(b) **Inter-clip reconstruction consistency of video tokenizers.** Existing video tokenizers [11, 52, 66] show the pixel-value inconsistency between short clips (16 frames). In contrast, Our tokenizer shows the temporally consistent reconstruction.

Figure 1. **Limitation of existing video tokenizers.** (a) Existing video tokenizers [11, 52, 66] are often not scalable to long videos because of excessive memory and computational demands. This is because they are trained to reconstruct all video frames at once, *i.e.*, a giant 3D array of pixels, which incurs a huge computation and memory burden in training especially when trained on long videos. For instance, PVDM-AE [66] becomes out-of-memory when trained to encode 128-frame videos when using a single NVIDIA 4090 24GB GPU. (b) As a result, existing tokenizers are typically trained to encode up to 16-frame videos and struggle to capture the temporal coherence of videos.

exploiting the temporal coherence of videos, CoordTok significantly reduces the number of tokens for encoding long videos compared to baselines. For instance, CoordTok encodes a 128-frame video with 128×128 resolution into only 1280 tokens, while baselines require 6144 or 8192 tokens to achieve similar encoding quality. We also show that efficient tokenization with CoordTok enables memory-efficient training of a diffusion transformer [27, 31] that can generate a 128-frame video at once. Finally, we provide an extensive analysis on the effect of various design choices.

We summarize the contributions of this paper below:

- We introduce CoordTok, a scalable video tokenizer that learns a mapping from coordinate-based representations to the corresponding patches of input videos.
- We show that CoordTok can leverage the temporal coherence of videos for tokenization, drastically reducing the number of tokens required for encoding long videos.
- We show that efficient video tokenization with CoordTok enables memory-efficient training of a diffusion transformer [27, 31] that can generate long videos at once.

2. Method

In this section, we present CoordTok, a scalable video tokenizer that can efficiently encode long videos. In a nutshell, CoordTok encodes a video into factorized triplane representations [22, 66] and learns a mapping from randomly sampled (x, y, t) coordinates to pixels from the corresponding patches. We provide the overview of CoordTok in Figure 2.

Problem setup Let $\mathbf{x}$ be a video and $\mathcal{D}$ be a dataset consisting of videos. Our goal is to train a video tokenizer that encodes a video $\mathbf{x} \in \mathcal{D}$ into tokens (or a low-dimensional latent vector) $\mathbf{z}$ and decodes $\mathbf{z}$ into $\mathbf{x}$. In particular, we want the tokenizer to be *efficient* so that it can encode videos into fewer number of tokens as possible but still can decode tokens to the original video $\mathbf{x}$ without loss of information.

2.1. Encoder

Given a video $\mathbf{x}$, we divide the video into non-overlapping space-time patches. We then add learnable positional embeddings and process them through a series of transformer layers [50] to obtain video features $\mathbf{e}$.

After that, we encode video features $\mathbf{e}$ into factorized triplane representations [4, 66], *i.e.*, $\mathbf{z} = [\mathbf{z}^{xy}, \mathbf{z}^{yt}, \mathbf{z}^{xt}]$, where the planes have the shape of $H' \times W'$, $W' \times T'$, and $H' \times T'$, respectively. Intuitively, $\mathbf{z}^{xy}$ captures the global content in $\mathbf{x}$ across time (*e.g.*, layout and appearance of the scene or object), $\mathbf{z}^{yt}$ and $\mathbf{z}^{xt}$ capture the underlying motion in $\mathbf{x}$ across two spatial axes (see Figure 8 for visualization). This design is efficient because it represents a video with three 2D latent planes instead of 3D latents widely used in prior approaches [11, 54, 63].

We implement our encoder based on the memory-efficient design of a recent 3D generation work [18] that introduces learnable embeddings and translates them to triplane representations. Specifically, we first introduce learnable embeddings $\mathbf{z}_0 = [\mathbf{z}_0^{xy}, \mathbf{z}_0^{yt}, \mathbf{z}_0^{xt}]$. We then process them through a series of cross-self attention layers, where each layer consists of (i) cross-attention layer that attends to the video features $\mathbf{e}$ and (ii) self-attention layer that attends to its own features. In practice, we split each learnable embedding into four smaller equal-sized embeddings. We then use them as inputs to the cross-self encoder, because we find it helps the model to use more computation by increasing the length of input sequence. Finally, we project the outputs into triplane representations to obtain $\mathbf{z} = [\mathbf{z}^{xy}, \mathbf{z}^{yt}, \mathbf{z}^{xt}]$.

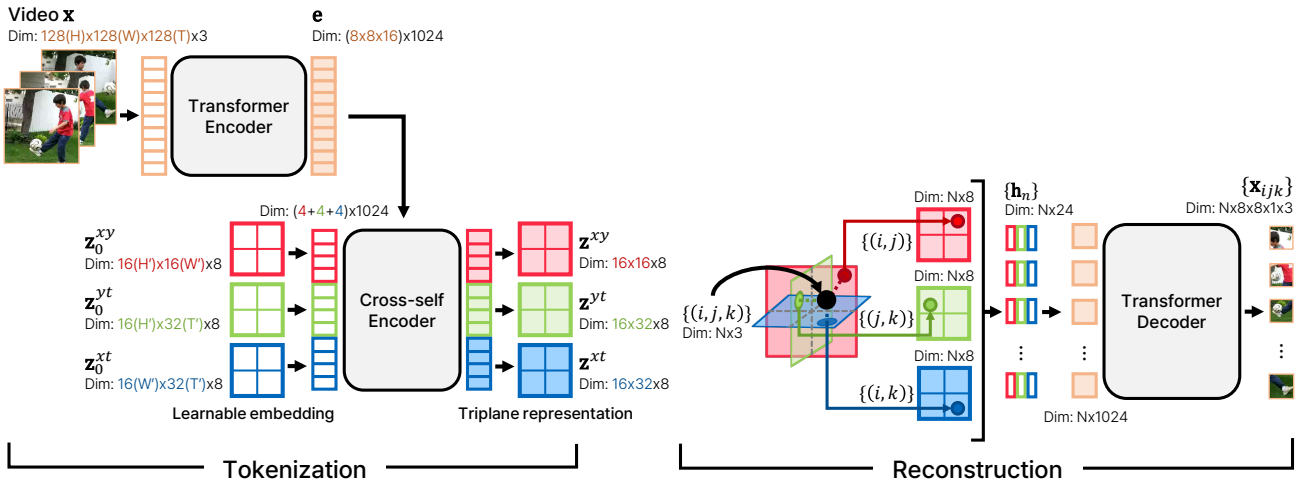


Figure 2. **Overview of CoordTok.** We design our encoder to encode a video $\mathbf{x}$ into factorized triplane representations $\mathbf{z} = [\mathbf{z}^{xy}, \mathbf{z}^{yt}, \mathbf{z}^{xt}]$ which can efficiently represent the video with three 2D latent planes. Given the triplane representations $\mathbf{z}$, our decoder learns a mapping from (x, y, t) coordinates to RGB pixels within the corresponding patches. In particular, we extract coordinate-based representations of N sampled coordinates by querying the coordinates from triplane representations via bilinear interpolation. Then the decoder aggregates and fuses information from different coordinates with self-attention layers and project outputs into corresponding patches. This design enables us to train tokenizers on long videos in a compute-efficient manner by avoiding reconstruction of entire frames at once.

2.2. Decoder

Given the triplane representation $\mathbf{z} = [\mathbf{z}^{xy}, \mathbf{z}^{yt}, \mathbf{z}^{xt}]$, we implement our decoder to reconstruct partial video during the training stage by learning a mapping from (i, j, k) coordinate to the pixels of the corresponding patch.

Input and target We use patch coordinates as inputs to the decoder and their corresponding patch RGB values as targets. Specifically, we first divide the video $\mathbf{x}$ into non-overlapping space-time patches. We note that the configuration of patches, *e.g.*, patch sizes, may differ from the one used in the video encoder. We then convert each patch index into the (i, j, k) coordinates representing the center position of the patch along each x, y, and t axis relative to the entire video $\mathbf{x}$, where $i, j, k \in [0, 1]$. Finally, we randomly sample N patches. We find that sampling only 3% of video patches can achieve strong performance (see Table 4 for the effect of sampling).

$$\begin{aligned} &\text{Input:} \quad [(i_1, j_1, k_1), \cdots, (i_N, j_N, k_N)] \\ &\text{Target:} \quad [\mathbf{x}_{i_1 j_1 k_1}, \cdots, \mathbf{x}_{i_N j_N k_N}] \end{aligned} \tag{1}$$

Coordinate-based representations As inputs to the transformer decoder, we use coordinate-based representations $\mathbf{h}$ that are obtained by querying each input coordinate from triplane representation via bilinear interpolation. Specifically, let (i, j, k) be one of sampled coordinates. We extract $\mathbf{h}^{xy}$ by querying (i, j) from $\mathbf{z}^{xy}$, $\mathbf{h}^{yt}$ by querying (j, k) from $\mathbf{z}^{yt}$, and $\mathbf{h}^{xt}$ by querying (i, k) from $\mathbf{z}^{xt}$. More specifically, let (l, m, n) be the indices in the triplane representation corresponding to (i, j, k), obtained using the floor function, *i.e.*, $(l, m, n) = (\lfloor iH' \rfloor, \lfloor jW' \rfloor, \lfloor kT' \rfloor)$. Then, coordinate-based representations are computed as follows:

$$\begin{aligned} \mathbf{h}^{xy} &= \text{Bilerp}((i,j); \mathbf{z}^{xy}_{lm}, \mathbf{z}^{xy}_{l,m+1}, \mathbf{z}^{xy}_{l+1,m}, \mathbf{z}^{xy}_{l+1,m+1}) \\ \mathbf{h}^{yt} &= \text{Bilerp}((j,k); \mathbf{z}^{yt}_{mn}, \mathbf{z}^{yt}_{m,n+1}, \mathbf{z}^{yt}_{m+1,n}, \mathbf{z}^{yt}_{m+1,n+1}) \\ \mathbf{h}^{xt} &= \text{Bilerp}((i,k); \mathbf{z}^{xt}_{ln}, \mathbf{z}^{xt}_{l,n+1}, \mathbf{z}^{xt}_{l+1,n}, \mathbf{z}^{xt}_{l+1,n+1}) \end{aligned} \tag{2}$$

where $(\mathbf{z}^{xy}_{lm}, \mathbf{z}^{yt}_{mn}, \mathbf{z}^{xt}_{ln})$ indicates the latent vector in $\mathbf{z}$ at indices (l, m, n), and $\text{Bilerp}(\cdot;\cdot)$ is the bilinear interpolation operation at the input coordinate between given vectors. We then concatenate them to get the coordinate-based representation of (i, j, k), *i.e.*, $\mathbf{h} := \text{Concat}(\mathbf{h}^{xy}, \mathbf{h}^{yt}, \mathbf{h}^{xt})$.

Patch reconstruction Given N coordinate-based representations $[\mathbf{h}_1, ..., \mathbf{h}_N]$, our decoder processes them through a series of self-attention layers, enabling each $\mathbf{h}_n$ to attend to other representations $\mathbf{h}_m$. This allows the decoder to aggregate and fuse the information from different coordinates. We then use a linear projection layer to process the output from each $\mathbf{h}_n$ to pixels of the corresponding patch $\mathbf{x}_{i_n j_n k_n}$. Finally, we update the parameters of our encoder and decoder to minimize an ℓ_2 loss between the reconstructed pixels and original pixels.

To further improve the quality of reconstructed videos, we introduce an additional fine-tuning phase where we train our tokenizer with both ℓ_2 loss and LPIPS loss [68]. Specifically, instead of sampling coordinates, we randomly sample a few frames and use all coordinates within the sampled

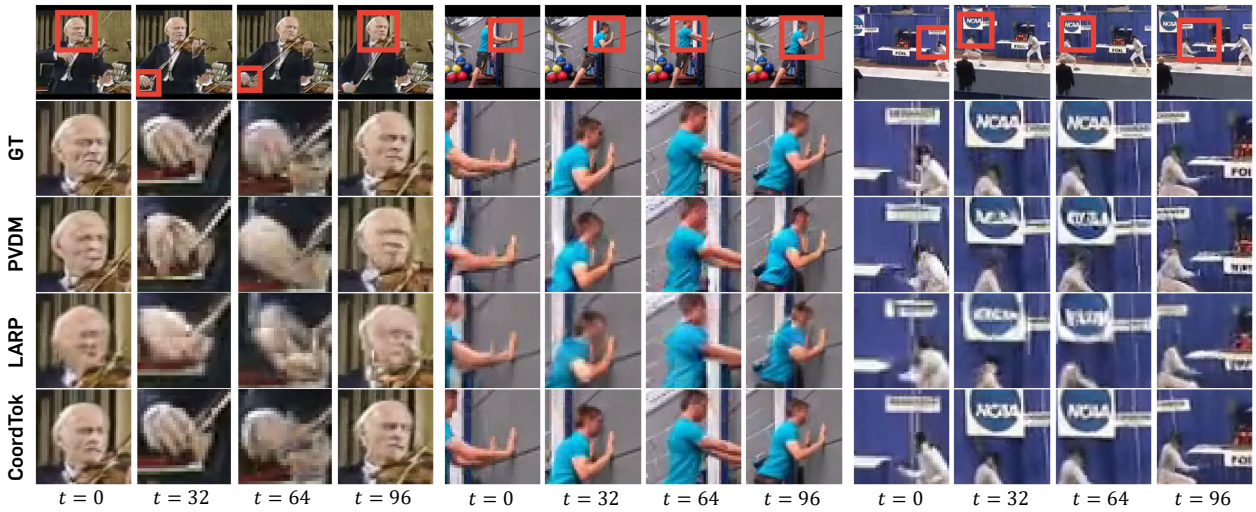


Figure 3. **128-frame, 128×128 resolution video reconstruction results** from CoordTok (Ours) and baselines [52, 66] trained on the UCF-101 dataset [42]. For each frame, we visualize the ground-truth (GT) and reconstructed pixels within the region highlighted in the red box, where CoordTok achieves noticeably better reconstruction quality than other baselines.

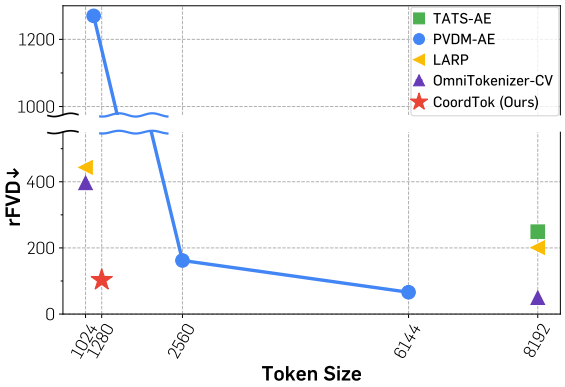


Figure 4. **CoordTok can efficiently encode long videos.** rFVD scores of video tokenizers, evaluated on 128-frame videos, with respect to the token size. ↓ indicates lower values are better.

frames for fine-tuning. This enables the tokenizer to compute and minimize LPIPS loss, which requires reconstructing the entire frame. While we find that sampling frames instead of coordinates from the beginning of the training is harmful due to the lack of diversity in training data (see Table 4), we find that fine-tuning with sampled frames improves the quality of reconstructed videos.

3. Experiments

We design experiments to investigate following questions:

- Can CoordTok efficiently encode long videos? Does encoding long videos lead to efficient video tokenization (Figures 3 and 4 and Table 1)
- Can CoordTok learn meaningful tokens that can be used for downstream tasks such as video generation? (Table 2) Can efficient video tokenization improve video generation models? (Table 3)
- What is the effect of various design choices? (Figure 6 and Table 4)

3.1. Experimental Setup

Implementation details We conduct all our experiments on the UCF-101 [42] dataset. Following the setup of prior works [11, 63], we use the train split of the UCF-101 dataset for training. For preprocessing videos, we resize and center-crop the frames to 128×128 resolution. We train our tokenizer using the AdamW optimizer [25] with a batch size of 256, where each sample is a randomly sampled 128-frame video. We use $N = 1024$ coordinates for main training and $N = 4096$ for fine-tuning. For the main experimental results, we train CoordTok for 1M iterations and further fine-tune it with LPIPS loss for 50k iterations. For analysis and ablation studies, we train CoordTok for 200k iterations and further fine-tune it with LPIPS loss for 10k iterations. For model configurations such as embedding dimension and number of layers, we mostly follow the architectures of vision transformers (ViTs; [7]). We provide more detailed implementation details in Appendix A.

Evaluation For evaluating the quality of reconstructed videos, we follow the setup of MAGVIT [63] that reports reconstruction Fréchet video distance (rFVD; [48]), peak signal-to-noise ratio (PSNR), LPIPS [68], and SSIM [55]. We use 10000 video clips of length 128 for evaluation. For evaluating the quality of generated videos, we follow the setup of StyleGAN-V [39] that reports FVD measured with 2048 video clips. We provide more details of evaluation metrics in Appendix B.

Table 1. **Reconstruction quality of image and video tokenizers.** We report metrics that measure the quality of reconstructed videos: PSNR, LPIPS, SSIM, and rFVD, computed using the 128×128 resolution videos reconstructed by image and video tokenizers evaluated on the UCF-101 dataset [42]. All models except CosmosTokenizer* [9] are trained on UCF-101. Total # tokens denotes the number of tokens required for encoding 128-frame videos. # Frames denotes number of frames in a video used for training tokenizers. ↓ and ↑ denotes whether lower or higher values are better, respectively.

Method	Token type	Total # tokens	# Frames	Reconstruction quality			
				PSNR↑	LPIPS↓	SSIM↑	rFVD↓
MaskGIT-AE [5]	Discrete	8192	1	21.4	0.139	0.667	447.1
TATS-AE [11]	Discrete	8192	16	23.2	0.213	0.792	249.4
MAGVIT-AE-L [63]	Discrete	8192	16	21.8	0.113	0.690	-
LARP [52]	Discrete	8192	16	24.3	0.142	0.806	201.3
OmniTokenizer-DV [53]	Discrete	8192	17	26.1	0.113	0.871	97.9
PVDM-AE [66]	Continuous	6144	16	26.5	0.120	0.859	66.5
OmniTokenizer-CV [53]	Continuous	8192	17	28.3	0.081	0.913	49.5
CosmosTokenizer-CV* [9]	Continuous	8192	17	28.5	0.119	0.905	87.8
LARP [52]	Discrete	1024	16	22.0	0.181	0.766	443.5
OmniTokenizer-DV [53]	Discrete	1024	17	22.2	0.201	0.703	509.0
PVDM-AE [66]	Continuous	1152	16	19.1	0.333	0.563	1270.1
OmniTokenizer-CV [53]	Continuous	1024	17	23.2	0.175	0.744	396.7
CosmosTokenizer-CV* [9]	Continuous	1024	17	24.0	0.220	0.774	519.6
CoordTok (Ours)	Continuous	1280	128	**28.6**	**0.066**	**0.892**	**102.9**

Table 2. **FVDs of video generation models** on the UCF-101 dataset (128-frame, 128×128 resolution). ↓ indicates lower values are better.

Method	FVD↓
MoCoGAN [47]	3679.0
+ StyleGAN2 [20]	2311.3
MoCoGAN-HD [46]	2606.5
DIGAN [65]	2293.7
StyleGAN-V [39]	1773.4
PVDM-L [66]	505.0
HVDM [21]	549.7
Latte-L/2 [10]	1901.8
CoordTok-SiT-L/2 (Ours)	**369.3**

Table 3. **Video generation efficiency.** We report time (s) and memory (GB) required for synthesizing a 128-frame video using a single NVIDIA 4090 24GB GPU. We use the DDIM sampler [41] with 200 sampling steps for PVDM-L and HVDM and use the Euler-Maruyama sampler [27] with 250 sampling steps for our method.

Method	Time (s)	Mem (GB)
TATS [11]	180.7	9.8
LARP [52]	114.3	**3.1**
PVDM-L [66]	116.9	4.0
HVDM [21]	52.1	3.9
Latte-L/2 [10]	21.4	**3.1**
CoordTok-SiT-L/2 (Ours)	**9.8**	4.5

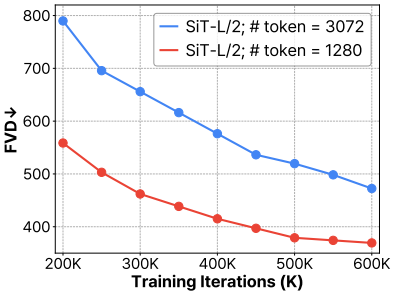
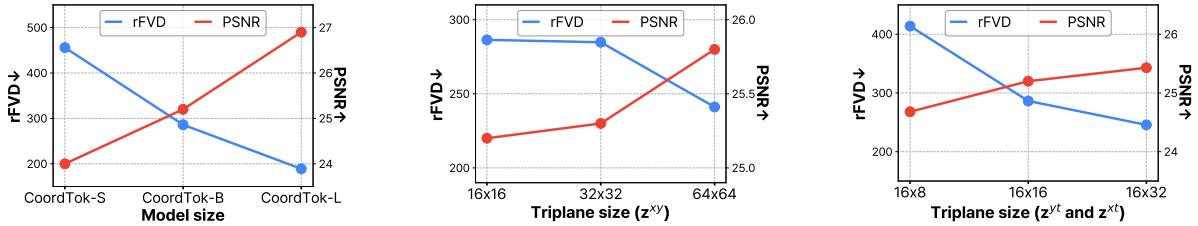


Figure 5. **Efficient video tokenization improves video generation.** We report FVDs of SiT-L/2 models trained upon CoordTok with token sizes of 1280 and 3072. ↓ indicates lower values are better.

3.2. Long video tokenization

Setup To investigate whether training CoordTok to encode long videos at once leads to efficient tokenization, we consider a setup where tokenizers encode 128-frame videos. Because existing tokenizers cannot encode such long videos at once, we split videos into multiple 16-frame video clips, use baseline tokenizers to encode each of them, and then concatenate the tokens from entire splits. For CoordTok, we train our tokenizer to encode 128-frame videos at once. We provide more details in Appendix A.

Baselines We mainly consider tokenizers used in recent image or video generation models as our baselines. We first consider MaskGIT-AE [5], an image tokenizer, as our baseline to evaluate the benefit of using video tokenizers for encoding videos. Moreover, we consider PVDM-AE [66], which encodes a video into factorized triplane representations and decodes all frames at once, as another baseline. Comparison with PVDM-AE enables us to evaluate the benefit of our decoder design because it shares the same latent structure with CoordTok. We further consider recent video tokenizers that encode videos into 3D latents, *i.e.*, TATS-AE [11], MAGVIT-AE-L [63], LARP [52], OmniTokenizer-DV [53], and OmniTokenizer-CV [53] as our baselines. For a fair comparison, we train all baselines from scratch on UCF-101 or use the model weights trained on UCF-101 following their official implementations. In addition, we compare CoordTok to CosmosTokenizer-CV [9], a state-of-the-art tokenizer, although it is not a directly comparable baseline because it is trained on a large-scale dataset. We provide more details of each baseline in Appendix C.



(a) Effect of Model size (b) Effect of Triplane size (spatial) (c) Effect of Triplane size (temporal)

Figure 6. **Analysis on the effect of (a) model size, (b) spatial dimensions of triplane representations, and (c) temporal dimensions of triplane representations.** For our main experiments, we use CoordTok-L with triplane representations of 16×16 spatial dimensions and 32 temporal dimensions. ↓ and ↑ denote whether lower or higher values are better, respectively.

Results For qualitative evaluation, we provide videos reconstructed by CoordTok and other baseline tokenizers in Figure 3. Notably, we find that CoordTok efficiently encodes 128-frame videos into only 1280 tokens. In contrast, baselines achieve significantly worse reconstruction quality when they use a similar number of tokens to CoordTok. For instance, CoordTok can encode 128-frame videos to 1280 tokens with a rFVD score of 103, while PVDM-AE achieves >1000 rFVD score when using 1152 tokens. This highlights the benefit of our decoder design, which enables the tokenizer to exploit the temporal coherence of long videos better for efficient tokenization. Moreover, Table 1 shows CoordTok outperforms baseline tokenizers across diverse metrics that assess the quality of reconstructed frames.

3.3. Long video generation

Setup To investigate whether CoordTok can encode long videos into meaningful tokens, we consider an unconditional video generation setup where we train a model to produce 128-frame videos. Videos of length 128 are often considered too long to be generated at once, so several works use techniques such as iterative generation [66] for generating long videos. However, because CoordTok can efficiently encode long videos, we train our model to generate 128-frame videos at once. Specifically, we encode 128-frame videos into 1280 tokens with CoordTok and train a SiT-L/2 model [27], a recent flow-based transformer model, for 600K iterations with a batch size of 64. We then use the model to generate 128-frame videos using the Euler-Maruyama sampler with 250 sampling steps. We provide more implementation details in Appendix A.

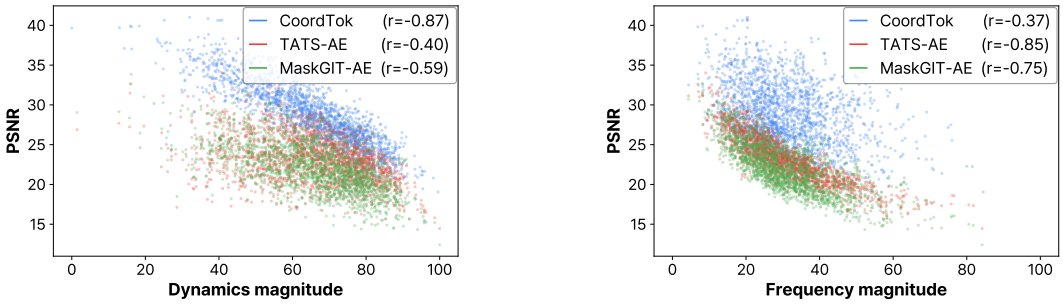
Baselines We consider recent video generation models that can generate 128-frame videos as baselines, *i.e.*, MoCoGAN [47], MoCoGAN-HD [46], DIGAN [65], StyleGAN-V [39], PVDM-L [66], HVDM [21], and Latte-L/2 [10]. We provide more details of each baseline in Appendix C.

Results Table 2 provides the quantitative evaluation of our model, *i.e.*, CoordTok-SiT-L/2, and other video generation models. We find that CoordTok-SiT-L/2 achieves the best FVD score of 369.3, outperforming previous baselines. This is an intriguing result considering that CoordTok-SiT-L/2 can generate 128-frame videos much faster than other baselines, as shown in Table 3. Moreover, to investigate whether efficient video tokenization improves video generation, we evaluate the FVD scores of SiT-L/2 models trained with CoordTok using token sizes of 1280 and 3072. Figure 5 shows that SiT-L/2 trained with the token size of 1280 achieves consistently low FVD scores, even though there is no significant difference in the reconstruction quality of CoordTok with 1280 and 3072 tokens (see Appendix D). This is likely because the reduced number of tokens makes it easier to train the SiT model. For qualitative evaluation, we provide videos from CoordTok-SiT-L/2 in Appendix F.

3.4. Analysis and ablation studies

Effect of model Size In Figure 6a, we investigate the scalability of CoordTok with respect to model sizes. We evaluate three variants of CoordTok: CoordTok-S, CoordTok-B, and CoordTok-L. Each variant has a different size for the encoder and decoder (see Appendix A for detailed model configurations). We find that the quality of reconstructed videos improves as the model size increases. For instance, CoordTok-B achieves a PSNR of 25.2 while CoordTok-L achieves a PSNR of 26.9.

Effect of triplane size In Figure 6b and Figure 6c, we investigate the effect of spatial and temporal dimensions in triplane representations. We evaluate CoordTok with varying spatial dimensions (16×16, 32×32, and 64×64), and varying temporal dimensions (8, 16, and 32). In general, we find that using larger planes improves the quality of reconstructed videos, as the model can better represent details within videos using more tokens. This result suggests there is a trade-off between the number of tokens and the reconstruction quality. In practice, we find reducing the spatial dimensions to 16×16 while using a high temporal dimension of 32 strikes a good balance, achieving good quality of reconstructed videos with a relatively low number of tokens.



(a) Effect of triplane representations.

(b) Effect of coordinate-based representations

Figure 7. **Analysis on the effect of (a) triplane representations and (b) coordinate-based representations.** (a) We measure the Pearson correlation r between the reconstruction quality and a dynamics metric that measures how dynamic each video is. A video with a larger dynamics magnitude indicates a more dynamic video. We find that the correlation is stronger for CoordTok compared to TATS-AE [11] and MaskGIT-AE [5], which encode videos into 3D latents. We hypothesize this is because it is difficult to decompose dynamic videos into contents ($\mathbf{z}^{xy}$) and motions ($\mathbf{z}^{yt}$, $\mathbf{z}^{xt}$). (b) We measure the Pearson correlation r between the reconstruction quality and a frequency metric that measures the fineness of video details [60]. A video with a larger frequency magnitude indicates a finer-grained video. In this case, we find that the correlation is weaker for CoordTok compared to other tokenizers. We hypothesize this is because CoordTok explicitly learns a mapping from each coordinate-based representation to pixels within the corresponding patch.

Effect of triplane representations We now examine the effect of one of our key design choices: encoding videos into triplane representations rather than 3D latents. We hypothesize that CoordTok may struggle to encode dynamic videos, as decomposing a video to its content ($\mathbf{z}^{xy}$) and motion components ($\mathbf{z}^{yt}$, $\mathbf{z}^{xt}$) becomes difficult. To investigate this, in Figure 7a, we provide a scatter plot where the x-axis represents a metric for video dynamics and the y axis represents the PSNR score. As a metric for video dynamics, we use the mean ℓ_2-distance between pixel values of consecutive frames (see Appendix B for more details). As expected, we find that the correlation between reconstruction quality and the magnitude of dynamics is strong (-0.87) for CoordTok, compared to the weaker correlations for TATS-AE (-0.40) and MaskGIT-AE (-0.59), both of which use 3D latent structures. This is one of the limitations of CoordTok, and addressing this by adopting techniques from video codecs, such as introducing multiple keyframes, could be an interesting future direction.

Effect of coordinate-based representations We further examine the effect of our design that trains using coordinate-based representations. Our hypothesis is that the reconstruction quality of CoordTok is less sensitive to how fine-grained each video is, because CoordTok learns a mapping from each coordinate to pixels. To investigate this, we measure the correlation between the PSNR score and a frequency metric proposed in Yan et al. [60] that utilizes a Sobel edge detection filter, where a larger frequency magnitude indicates a finer-grained video (see Appendix B for details). As shown in Figure 7b, the correlation between reconstruction quality and the frequency metric is weak (-0.37) for CoordTok, compared to stronger correlations for TATS-AE (-0.85) and MaskGIT-AE (-0.75).

Table 4. **Effect of sampling**. We report rFVD and the maximum batch size (Max BS) measured with a single NVIDIA 4090 24GB GPU, with different sampling schemes. Random patch uses center coordinates of randomly selected patches for training, while Random frame uses all coordinates from a few randomly sampled frames for training. Ratio (%) indicates the proportion of sampled coordinates relative to all possible coordinates within a video. ↓ indicates lower values are better.

Sampling	Ratio (%)	rFVD↓	Max BS
Random frame	3.125	479	13
Random patch	1.563	401	21
Random patch	3.125	238	13

Effect of sampling We investigate two coordinate sampling schemes: (i) Random patch, which uses center coordinates of randomly sampled patches, and (ii) Random frame, which uses all coordinates from a few randomly sampled frames. As shown in Table 4, Random patch outperforms Random frame when sampling the same number of coordinates. We hypothesize this is because Random frame fails to provide the tokenizer with sufficiently diverse training data. For instance, sampling 3.125% of video patches corresponds to sampling only 4 frames out of 128 in the Random frame scheme. In contrast, Random patch uniformly samples patches from all 128 frames, which helps provide more diverse training data. For Random patch, we find that sampling fewer coordinates reduces the training memory requirement but also degrades performance.

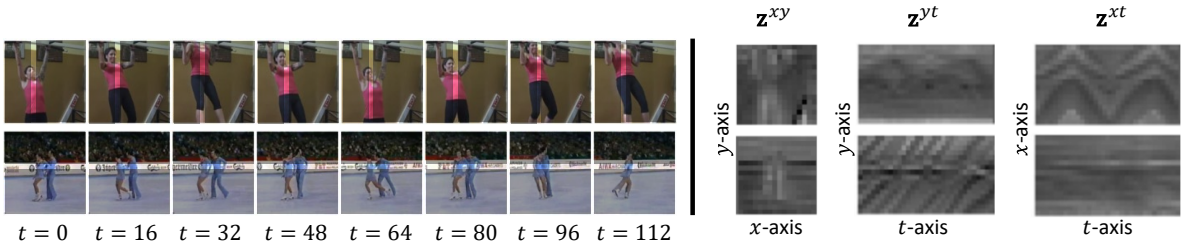


Figure 8. **Illustration of factorized triplane representations** $\mathbf{z} = [\mathbf{z}^{xy}, \mathbf{z}^{yt}, \mathbf{z}^{xt}]$ of CoordTok trained on the UCF-101 dataset [42]. We note that $\mathbf{z}^{xy}$ captures the global content in the video across time, *e.g.*, layout and appearance of the scene or object, and $\mathbf{z}^{yt}$, $\mathbf{z}^{xt}$ capture the underlying motion in the video across two spatial axes.

4. Related Work

Video tokenization Many recent works have explored the idea of using video tokenizers to encode videos into low-dimensional latent tokens. Initial attempts proposed to directly use image tokenizers for videos [8, 33, 49] via frame-wise compression. However, this approach overlooks the temporal coherence of videos, resulting in inefficient compression. Thus, recent works have proposed to train a tokenizer specialized for videos [1, 2, 11, 17, 53, 54, 58, 59, 61, 63, 64]. They typically extend image tokenizers by replacing spatial layers with spatiotemporal layers (*e.g.*, 2D convolutional layers to 3D convolution layers). More recent works have introduced efficient tokenization schemes with careful consideration of redundancy in video data. For instance, several works proposed to encode videos into factorized triplane representations [21, 66, 67], and another line of works proposed an adaptive encoding scheme that utilizes the redundancy of videos for tokenization [52, 60]. However, they still train the tokenizer through reconstruction of entire video frames, so training is only possible with short video clips split from the original long videos. Our work introduces a video tokenizer that can directly handle much longer video clips by removing the need for a decoder to reconstruct entire video frames during training. By capturing the global information present in long videos, we show that our tokenizer achieves more effective tokenization.

Latent video generation Instead of modeling distributions of complex and high-dimensional video pixels, most recent video generation models focus on learning the latent distribution induced by video tokenizers, as it can dramatically reduce memory and computation bottlenecks. One approach involves training autoregressive models [11, 23, 58] in a discrete token space [33, 49]. Another line of research [51, 59, 62] also considers discrete latent space but has trained masked generative transformer (MaskGiT; [5]) for generative modeling. Finally, many recent works [1, 13, 15, 21, 26, 37, 66, 70] have trained diffusion models [16, 40] in continuous latent space, inspired by the success of latent diffusion models in the image domain [35]. Despite their efforts, the models are typically limited to processing only short video clips at a time (usually 16-frame clips), which makes it difficult for the model to generate longer videos. In this paper, we significantly improve the limited contextual length of latent video generation models by introducing an efficient video tokenizer.

5. Conclusion

In this paper, we have presented CoordTok, a scalable video tokenizer that learns a mapping from coordinate-based representations to the corresponding patches of input videos. CoordTok is built upon our intuition that training a tokenizer directly on long videos would enable the tokenizer to leverage the temporal coherence of videos for efficient tokenization. Our experiments show that CoordTok can encode long videos using far fewer number of tokens than existing baselines. We also find that this efficient video tokenization enables memory-efficient training of video generation models that can generate long videos at once. We hope that our work further facilitates future researches on designing scalable video tokenizers and efficient video generation models.

Limitations and future directions One limitation of our work is that our tokenizer struggles more with dynamic videos than with static videos, as shown in Figure 7. We hypothesize this is due to the difficulty of learning to decompose dynamic videos into global content and motion. One interesting future direction could involve introducing multiple content planes across the temporal dimension. Moreover, future work may introduce an adaptive method for deciding the number of such content planes based on how dynamic each video is, similar to techniques in video codecs [14, 30, 44, 56] or an adaptive encoding scheme designed for a recent video tokenizer [60]. Lastly, we are excited about scaling up our tokenizer to longer videos from larger datasets and evaluating it on challenging downstream tasks such as long video understanding and generation.

Acknowledgements

This work was supported by Institute for Information & communications Technology Promotion(IITP) grant funded by the Korea government(MSIT) (No.RS-2019-II190075 Artificial Intelligence Graduate School Program(KAIST); No.RS-2021-II212068, Artificial Intelligence Innovation Hub) and Samsung Electronics Co., Ltd (IO201211-08107-01). PA holds concurrent appointments as a Professor at UC Berkeley and as an Amazon Scholar. This paper describes work performed at UC Berkeley and is not associated with Amazon. YS is supported in part by Multidisciplinary University Research Initiative (MURI) award by the Army Research Office (ARO) grant No. W911NF-23-1-0277. We thank NVIDIA for providing compute resources through the NVIDIA Academic DGX Grant.

References

[1] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2023. 1, 8

[2] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. *OpenAI Blog*, 2024. 1, 8

[3] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2017. 2

[4] Eric R Chan, Connor Z Lin, Matthew A Chan, Koki Nagano, Boxiao Pan, Shalini De Mello, Orazio Gallo, Leonidas J Guibas, Jonathan Tremblay, Sameh Khamis, et al. Efficient geometry-aware 3d generative adversarial networks. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 2

[5] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T Freeman. MaskGIT: Masked generative image transformer. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 5, 7, 8, 2, 3

[6] Zeyuan Chen, Yinbo Chen, Jingwen Liu, Xingqian Xu, Vidit Goel, Zhangyang Wang, Humphrey Shi, and Xiaolong Wang. VideoINR: Learning video implicit neural representation for continuous space-time super-resolution. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 1

[7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. 4

[8] Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2021. 8, 2

[9] NVIDIA et al. Cosmos world foundation model platform for physical ai. *arXiv preprint*, 2025. 5

[10] Xin Ma et al. Latte: Latent diffusion transformer for video generation. *arXiv preprint*, 2024. 5, 6, 3

[11] Songwei Ge, Thomas Hayes, Harry Yang, Xi Yin, Guan Pang, David Jacobs, Jia-Bin Huang, and Devi Parikh. Long video generation with time-agnostic vqgan and time-sensitive transformer. In *European Conference on Computer Vision*, 2022. 1, 2, 4, 5, 7, 8, 3

[12] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014. 3

[13] Agrim Gupta, Lijun Yu, Kihyuk Sohn, Xiuye Gu, Meera Hahn, Li Fei-Fei, Irfan Essa, Lu Jiang, and José Lezama. Photorealistic video generation with diffusion models. In *European Conference on Computer Vision*, 2024. 8

[14] Jingning Han, Bohan Li, Debargha Mukherjee, Ching-Han Chiang, Adrian Grange, Cheng Chen, Hui Su, Sarah Parker, Sai Deng, Urvang Joshi, et al. A technical overview of av1. *Proceedings of the IEEE*, 109(9):1435–1462, 2021. 8

[15] Yingqing He, Tianyu Yang, Yong Zhang, Ying Shan, and Qifeng Chen. Latent video diffusion models for high-fidelity long video generation. *arXiv preprint arXiv:2211.13221*, 2022. 8

[16] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020. 8

[17] Wenyi Hong, Ming Ding, Wendi Zheng, Xinghan Liu, and Jie Tang. CogVideo: Large-scale pretraining for text-to-video generation via transformers. In *International Conference on Learning Representations*, 2023. 8

[18] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. LRM: Large reconstruction model for single image to 3D. In *International Conference on Learning Representations*, 2024. 1, 2

[19] Heewoo Jun and Alex Nichol. Shap-e: Generating conditional 3d implicit functions. *arXiv preprint arXiv:2305.02463*, 2023. 1

[20] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2020. 5, 3

[21] Kihong Kim, Haneol Lee, Jihye Park, Seyeon Kim, Kwanghee Lee, Seungryong Kim, and Jaejun Yoo. Hybrid video diffusion models with 2d triplane and 3d wavelet representation. In *European Conference on Computer Vision*, 2024. 1, 5, 6, 8, 3

[22] Subin Kim, Sihyun Yu, Jaeho Lee, and Jinwoo Shin. Scalable neural video representations with learnable positional features. In *Advances in Neural Information Processing Systems*, 2022. 1, 2

[23] Dan Kondratyuk, Lijun Yu, Xiuye Gu, José Lezama, Jonathan Huang, Grant Schindler, Rachel Hornung, Vighnesh Birodkar, Jimmy Yan, Ming-Chang Chiu, et al.

Videopoet: A large language model for zero-shot video generation. In *International Conference on Machine Learning*, 2023. 8

[24] Minghua Liu, Chao Xu, Haian Jin, Linghao Chen, Mukund Varma T, Zexiang Xu, and Hao Su. One-2-3-45: Any single image to 3d mesh in 45 seconds without per-shape optimization. In *Advances in Neural Information Processing Systems*, 2024. 1

[25] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2018. 4, 1

[26] Haoyu Lu, Guoxing Yang, Nanyi Fei, Yuqi Huo, Zhiwu Lu, Ping Luo, and Mingyu Ding. Vdt: General-purpose video diffusion transformers via mask modeling. In *International Conference on Learning Representations*, 2024. 8

[27] Nanye Ma, Mark Goldstein, Michael S Albergo, Nicholas M Boffi, Eric Vanden-Eijnden, and Saining Xie. SiT: Exploring flow and diffusion-based generative models with scalable interpolant transformers. In *European Conference on Computer Vision*, 2024. 2, 5, 6, 1

[28] Detlev Marpe, Thomas Wiegand, and Gary J Sullivan. The h. 264/mpeg4 advanced video coding standard and its applications. *IEEE communications magazine*, 2006. 1

[29] Takeru Miyato, Bernhard Jaeger, Max Welling, and Andreas Geiger. GTA: A geometry-aware attention mechanism for multi-view transformers. In *International Conference on Learning Representations*, 2024. 1

[30] Debargha Mukherjee, Jingning Han, Jim Bankoski, Ronald Bultje, Adrian Grange, John Koleszar, Paul Wilkins, and Yaowu Xu. A technical overview of vp9—the latest open-source video codec. *SMPTE Motion Imaging Journal*, 2015. 1, 8

[31] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE International Conference on Computer Vision*, 2023. 2

[32] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017. 2

[33] Ali Razavi, Aaron Van den Oord, and Oriol Vinyals. Generating diverse high-fidelity images with VQ-VAE-2. In *Advances in Neural Information Processing Systems*, 2019. 8

[34] Karel Rijkse. H. 263: Video coding for low-bit-rate communication. *IEEE Communications magazine*, 1996. 1

[35] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 8

[36] Karen Simonyan. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015. 2

[37] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, et al. Make-a-video: Text-to-video generation without text-video data. In *International Conference on Learning Representations*, 2023. 8

[38] Vincent Sitzmann, Julien Martel, Alexander Bergman, David Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, 2020. 3

[39] Ivan Skorokhodov, Sergey Tulyakov, and Mohamed Elhoseiny. StyleGAN-V: A continuous video generator with the price, image quality and perks of StyleGAN2. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 4, 5, 6, 2, 3

[40] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, 2015. 8

[41] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021. 5

[42] K Soomro. UCF101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012. 1, 4, 5, 8, 2, 6

[43] Gary J Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. Overview of the high efficiency video coding (hevc) standard. *IEEE Transactions on circuits and systems for video technology*, 2012. 1

[44] Vivienne Sze, Madhukar Budagavi, and Gary J Sullivan. High efficiency video coding (hevc). In *Integrated circuit and systems, algorithms and architectures*, page 40. Springer, 2014. 8

[45] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024. 1

[46] Yu Tian, Jian Ren, Menglei Chai, Kyle Olszewski, Xi Peng, Dimitris N Metaxas, and Sergey Tulyakov. A good image generator is what you need for high-resolution video synthesis. In *International Conference on Learning Representations*, 2021. 5, 6, 3

[47] Sergey Tulyakov, Ming-Yu Liu, Xiaodong Yang, and Jan Kautz. MoCoGAN: Decomposing motion and content for video generation. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2018. 5, 6, 3

[48] Thomas Unterthiner, Sjoerd van Steenkiste, Karol Kurach, Raphaël Marinier, Marcin Michalski, and Sylvain Gelly. FVD: A new metric for video generation, 2019. 4, 2

[49] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017. 8

[50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. 2

[51] Ruben Villegas, Mohammad Babaeizadeh, Pieter-Jan Kindermans, Hernan Moraldo, Han Zhang, Mohammad Taghi Saffar, Santiago Castro, Julius Kunze, and Dumitru Erhan. Phenaki: Variable length video generation from open domain textual descriptions. In *International Conference on Learning Representations*, 2022. 8

[52] Hanyu Wang, Saksham Suri, Yixuan Ren, Hao Chen, and Abhinav Shrivastava. LARP: Tokenizing videos with a learned autoregressive generative prior. *arXiv preprint arXiv:2410.21264*, 2024. 1, 2, 4, 5, 8, 3

[53] Junke Wang, Yi Jiang, Zehuan Yuan, Binyue Peng, Zuxuan Wu, and Yu-Gang Jiang. OmniTokenizer: A joint image-video tokenizer for visual generation. In *Advances in Neural Information Processing Systems*, 2024. 5, 8, 2, 3

[54] Xinlong Wang, Xiaosong Zhang, Zhengxiong Luo, Quan Sun, Yufeng Cui, Jinsheng Wang, Fan Zhang, Yueze Wang, Zhen Li, Qiying Yu, et al. Emu3: Next-token prediction is all you need. *arXiv preprint arXiv:2409.18869*, 2024. 1, 2, 8

[55] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 2004. 4, 2

[56] Thomas Wiegand, Gary J Sullivan, Gisle Bjontegaard, and Ajay Luthra. Overview of the h. 264/avc video coding standard. *IEEE Transactions on circuits and systems for video technology*, 13(7):560–576, 2003. 8

[57] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision*, 2018. 2

[58] Wilson Yan, Yunzhi Zhang, Pieter Abbeel, and Aravind Srinivas. Videogpt: Video generation using vq-vae and transformers. *arXiv preprint arXiv:2104.10157*, 2021. 8

[59] Wilson Yan, Danijar Hafner, Stephen James, and Pieter Abbeel. Temporally consistent transformers for video generation. In *International Conference on Machine Learning*, 2023. 8

[60] Wilson Yan, Matei Zaharia, Volodymyr Mnih, Pieter Abbeel, Aleksandra Faust, and Hao Liu. ElasticTok: Adaptive tokenization for image and video. *arXiv preprint arXiv:2410.08368*, 2024. 7, 8, 2

[61] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. CogvideoX: Text-to-video diffusion models with an expert transformer. *arXiv preprint arXiv:2408.06072*, 2024. 8

[62] Jaehoon Yoo, Semin Kim, Doyup Lee, Chiheon Kim, and Seunghoon Hong. Towards end-to-end generative modeling of long videos with memory-efficient bidirectional transformers. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2023. 8

[63] Lijun Yu, Yong Cheng, Kihyuk Sohn, José Lezama, Han Zhang, Huiwen Chang, Alexander G Hauptmann, Ming-Hsuan Yang, Yuan Hao, Irfan Essa, et al. Magvit: Masked generative video transformer. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2023. 1, 2, 4, 5, 8, 3

[64] Lijun Yu, José Lezama, Nitesh B Gundavarapu, Luca Versari, Kihyuk Sohn, David Minnen, Yong Cheng, Vighnesh Birodkar, Agrim Gupta, Xiuye Gu, et al. Language model beats diffusion–tokenizer is key to visual generation. In *International Conference on Learning Representations*, 2024. 1, 8

[65] Sihyun Yu, Jihoon Tack, Sangwoo Mo, Hyunsu Kim, Junho Kim, Jung-Woo Ha, and Jinwoo Shin. Generating videos with dynamics-aware implicit generative adversarial networks. In *International Conference on Learning Representations*, 2022. 5, 6, 3

[66] Sihyun Yu, Kihyuk Sohn, Subin Kim, and Jinwoo Shin. Video probabilistic diffusion models in projected latent space. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2023. 1, 2, 4, 5, 6, 8, 3

[67] Sihyun Yu, Weili Nie, De-An Huang, Boyi Li, Jinwoo Shin, and Anima Anandkumar. Efficient video diffusion models via content-frame motion-latent decomposition. In *International Conference on Learning Representations*, 2024. 1, 8

[68] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2018. 3, 4, 2

[69] Chunting Zhou, Lili Yu, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model. *arXiv preprint arXiv:2408.11039*, 2024. 1

[70] Daquan Zhou, Weimin Wang, Hanshu Yan, Weiwei Lv, Yizhe Zhu, and Jiashi Feng. Magicvideo: Efficient video generation with latent diffusion models. *arXiv preprint arXiv:2211.11018*, 2022. 8

Efficient Long Video Tokenization via Coordinate-based Patch Reconstruction

Supplementary Material

A. Implementation Details

A.1. Long video tokenization

We train CoordTok via AdamW optimizer [25] with a constant learning rate of 10^{-4}, $(\beta_1, \beta_2) = (0.9, 0.999)$, and weight decay 0.001. We use a batch size of 256, where each sample is a randomly sampled 128-frame video. CoordTok is trained in two stages: main training and fine-tuning. In the main training stage, we reconstruct $N = 1024$ randomly sampled coordinates and update the model using ℓ_2 loss. In the fine-tuning stage, we reconstruct 16 randomly sampled frames (i.e., $N = 4096$ coordinates) and update the model using a combination of ℓ_2 loss and LPIPS loss with equal weights. To speed up training, we use mixed-precision (fp16). For the main experimental results, we train CoordTok for 1M iterations and fine-tune it for 50K iterations. For analysis and ablation studies, we train CoordTok for 200K iterations and fine-tune it for 10K iterations.

Architecture CoordTok consists of a *transformer encoder* that extracts video features from raw videos, a *cross-self encoder* that processes video features into triplane representations via cross-attention between learnable parameters and video features, and a *transformer decoder* that learns a mapping from coordinate-based representations into corresponding patches. In what follows, we describe each component in detail.

- **Transformer encoder** consists of a Conv3D patch embedding, learnable positional embedding, and transformer layers, where each transformer layer comprises self-attention and feed-forward layers.
- **Cross-self encoder** consists of plane-wise Conv2D patch embeddings, transformer layers, and plane-wise linear projectors, where each transformer layer comprises cross-attention, self-attention, and feed-forward layers.
- **Transformer decoder** consists of linear patch embedding, learnable positional embedding, transformer layers, and a linear projector, where each transformer layer comprises self-attention and feed-forward layers.

We provide the detailed architecture configurations for each model size in Table 5.

Table 5. Model configurations of CoordTok for each model size.

Model size	Module	#layers	Hidden dim.	#heads
Large	Transformer Encoder	8	1024	16
	Cross-self Encoder	24	1024	16
	Transformer Decoder	24	1024	16
Base	Transformer Encoder	8	768	12
	Cross-self Encoder	12	768	12
	Transformer Decoder	12	768	12
Small	Transformer Encoder	8	512	8
	Cross-self Encoder	8	512	8
	Transformer Decoder	8	512	8

A.2. Long video generation

We implement CoordTok-SiT-L/2 based on the original SiT implementation [27]. The inputs of SiT-L/2 are the normalized triplane representation obtained by tokenizing video clips of length 128 with CoordTok. To normalize the triplane representation, we randomly sample 2048 video clips of length 128 and calculate the mean and standard deviation for each plane. We train SiT-L/2 via AdamW optimizer [25] with a constant learning rate of 10^{-4}, $(\beta_1, \beta_2) = (0.9, 0.999)$, and no weight decay. We use a batch size of 64. We train the model for 600K iterations and we update an EMA model with a momentum parameter 0.9999.

Architecture We use the same structure as SiT, except that our patch embedding and final projection layers are implemented separately for each plane. To train the unconditional video generation model, we assume the number of classes as 1, and we set the class dropout ratio to 0. We provide the detailed architecture configurations in Table 6.

Table 6. Model configurations of CoordTok-SiT-L/2.

	SiT-L/2, #token = 1280	SiT-L/2, #token = 3072
Input dim. ($\mathbf{z}^{xy}$)	16×16×8	32×32×8
Input dim. ($\mathbf{z}^{yt}$)	16×32×8	32×32×8
Input dim. ($\mathbf{z}^{xt}$)	16×32×8	32×32×8
# layers	24	24
Hidden dim.	1024	1024
# heads	16	16

Sampling For sampling, we use the Euler-Maruyama sampler with 250 sampling steps and a diffusion coefficient $w_t = \sigma_t$. We use the last step of the SDE sampler as 0.04.

B. Evaluation Details

B.1. Long video reconstruction

For our CoordTok, we tokenize and reconstruct 128-frame videos all at once. Specifically, we encode the video into a triplane representation and then reconstruct the video by passing all patch coordinates through the transformer decoder at once. In contrast, the baselines can only handle

videos of much shorter lengths (e.g., 16 frames for PVDM-AE [66]). Therefore, to evaluate the reconstruction quality of 128-frame videos for the baselines, we split the videos into short clips and tokenize and reconstruct them. To be specific, we first split a 128-frame video into shorter clips suitable for each tokenizer. We then tokenize and reconstruct each short clip individually using the tokenizer. Finally, we concatenate all the reconstructed short clips to obtain the 128-frame video.

For evaluating the quality of reconstructed videos, we follow the setup of MAGVIT [63]. We randomly sample 10000 video clips of length 128, and then measure the reconstruction quality using the metrics as follows:

- **rFVD** [48] measures the feature distance between the distributions of real and reconstructed videos. It uses the I3D network [3] to extract features, and it computes the distance based on the assumption that both feature distributions are multivariate Gaussian. Specifically, we compute the rFVD score on video clips of length 128.
- **PSNR** measures the similarity between pixel values of real and reconstructed images using the mean squared error. For videos, we compute the PSNR score for each frame and then average these frame-wise PSNR scores.
- **LPIPS** [68] measures the perceptual similarity between real and reconstructed images by computing the feature distance using a pre-trained VGG network [36]. It aggregates the distance of features extracted from various layers. For videos, we compute the LPIPS score for each frame and then average these frame-wise LPIPS scores.
- **SSIM** [55] measures the structural similarity between real and reconstructed images by comparing luminance, contrast, and structural information. For videos, we compute the SSIM score for each frame and then average these frame-wise SSIM scores.

B.2. Long video generation

For sname-SiT-L/2, we generate the tokens corresponding to a 128-frame video all at once and then decode these tokens using CoordTok. In contrast, baselines iteratively generate 128-frame videos. For instance, PVDM and HVDM generate the next 16-frame video conditioned on the previously generated 16-frame video clip.

For evaluating the quality of generated videos, we strictly follow the setup of StyleGAN-V [39] that calculates the FVD scores [48] between the distribution of real and generated videos. To be specific, we use 2048 video clips of length 128 for each distribution, where the real videos are sampled from the dataset used to train generation models (*i.e.*, the UCF-101 dataset [42]).

B.3. Analysis

- **Dynamics magnitude** To measure how dynamic each video is, we use the pixel value differences between consecutive frames. To be specific, we compute the dynamics magnitude for each pair of consecutive frames, calculate the mean of these values, and then take the logarithm. Here, dynamics magnitude of two frames f^1 and f^2 of resolution $H \times W$ can be defined as follows:

$$d(f^1, f^2) = \frac{1}{HW} \sum_{h=1}^{H} \sum_{w=1}^{W} d_2(f^1_{hw}, f^2_{hw}), \qquad (3)$$

  where f^i_{hw} denotes the RGB values at coordinates (h, w) of frame f^i and d_2 denotes ℓ_2-distance of RGB pixel values. In Figure 7a, we standardize the video dynamics score into a range of 0 to 100.
- **Frequency magnitude** To measure the frequency magnitude, we use the metric proposed in Yan et al. [60] that utilizes a Sobel edge detection filter. To be specific, to get the frequency magnitude, we apply both horizontal and vertical Sobel filters to each frame to compute the gradient magnitude at each pixel. We then calculate the average of these magnitudes across all pixels.

C. Baselines

C.1. Long video reconstruction

We describe the main idea of baseline methods that we used for the evaluation. We also provide the shape of tokens of baselines in Table 7.

- **MaskGiT-AE** [5] uses 2D VQ-GAN [8] that encodes an image into a 2D discrete tokens.
- **TATS-AE** [11] introduces 3D-VQGAN that compresses a 16-frame video clip both temporally and spatially into 3D discrete tokens.
- **MAGVIT-AE-L** [63] also introduces 3D-VQGAN but improves architecture design (*e.g.*, uses deeper 3D discriminator rather than two shallow discriminators for 2D and 3D separately, uses group normalization [57] and Swish activation [32]) and scales up the model size.
- **PVDM-AE** [66] encodes a 16-frame video clip into factorized triplane representations.
- **LARP** [52] encodes videos into 1D arrays by utilizing a next-token prediction model as a prior model.
- **OmniTokenizer-DV** [53] introduces image-video joint VQGAN that compresses a 17-frame video clip into 3D discrete tokens with more advanced architecture design (*e.g.*, uses both 2D and 3D patch embedding layers to support both image and video tokenization, uses transformer backbone with causal attention layers).
- **OmniTokenizer-CV** [53] uses the same architecture design as OmniTokenizer-DV, but replaces the VQ loss with KL loss so that it compresses a 17-frame video clip into 3D continuous latent vectors.

Table 7. Token shapes of video tokenization baselines

Method	Input shape	Token shape
MaskGiT-AE [5]	128×128×3	8×8
TATS-AE [11]	16×128×128×3	4×16×16
MAGVIT-AE-L [63]	16×128×128×3	4×16×16
PVDM-AE [66]	16×128×128×3	(16×16) × 3
LARP [52]	16×128×128×3	1024
OmniTokenizer [53]	(1+16)×128×128×3	(1+4)×16×16

C.2. Long video generation

We describe the main idea of baseline methods that we used for the evaluation.

- **MoCoGAN** [47] proposes a video generative adversarial network (GAN; [12]) that has a separate content generator and an autoregressive motion generator for generating videos.
- **MoCoGAN-HD** [46] also proposes a video GAN with motion-content decomposition but uses a strong pre-trained image generator (StyleGAN2 [20]) for a high-resolution image synthesis.
- **DIGAN** [65] interprets videos as implicit neural representation (INR; [38]) and trains GANs to generate such INR parameters.
- **StyleGAN-V** [39] also introduces an INR-based video GAN with a computation-efficient discriminator.
- **PVDM-L** [66] proposes a latent video diffusion model that generates videos in a projected triplane latent space.
- **HVDM** [21] proposes a latent video diffusion model that generates videos with 2D triplane and 3D wavelet representation.
- **Latte-L/2** [10] proposes a latent video diffusion transformer that generates video by processing latent vectors with alternating spatial and temporal attention layers.

D. Additional Analysis

Computational costs We provide the GPU memory usage during training in Figure 1a, and FLOPs during training in Figure 9. We find that our decoder design allows the efficient long video tokenization in terms of both GPU memory and FLOPs.

Analysis on the number of tokens We provide the reconstruction quality of CoordTok with 1280 and 3072 tokens in Table 8. Although there is no significant difference in the reconstruction quality between CoordTok with token sizes of 1280 and 3072, training SiT-L/2 with the 1280 tokens results in substantially better generation quality (see Section 3.3).

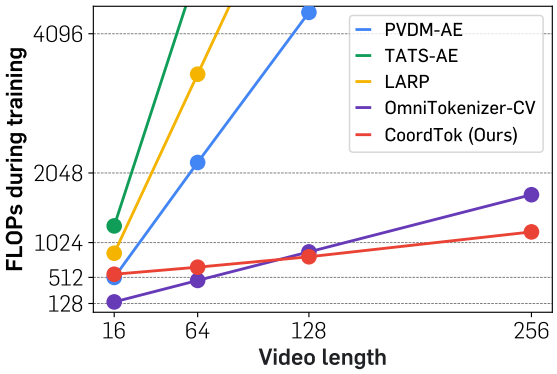


Figure 9. FLOPs when training video tokenizers on 128×128 resolution videos with varying lengths.

Table 8. Reconstruction quality of CoordTok with varying number of token sizes, evaluated on 128-frame videos. ↓ and ↑ denotes whether lower or higher values are better, respectively.

#tokens	rFVD↓	PSNR↑	LPIPS↓	SSIM↑
1280	102.9	28.6	0.066	0.892
3072	**100.5**	**28.7**	**0.065**	**0.894**

Analysis on the effect of LPIPS fine-tuning In Table 9, we investigate the effect of the additional fine-tuning phase, where we train CoordTok with both ℓ_2 loss and LPIPS loss [68] for 50K iterations after training CoordTok with ℓ_2 loss for 1M iterations. We find that fine-tuning phase improves the perceptual quality (*i.e.*, rFVD score: 188.3 → 102.9, and LPIPS score: 0.141 → 0.066), but degrades the pixel-level reconstruction quality (*i.e.*, PSNR: 30.3 → 28.6, and SSIM: 0.905 → 0.892).

Table 9. Effect of LPIPS fine-tuning phase for CoordTok. ↓ and ↑ denotes whether lower or higher values are better, respectively.

Phase	Iters	loss	rFVD↓	PSNR↑	LPIPS↓	SSIM↑
1	1M	ℓ_2	186.3	**30.3**	0.141	**0.905**
2	+50K	ℓ_2+LPIPS	**102.9**	28.6	**0.066**	0.892

E. Additional Quantitative Results

16-frame reconstruction quality To further evaluate the quality of reconstructed videos from tokenizers, we report the rFVD score on video clips of length 16 for the CoordTok and other tokenizers with varying number of token sizes in Figure 10. For evaluation, we use 10000 video clips of length 128, which are also used to measure the rFVD score on 128-frame videos. We split each 128-frame video into 16 non-overlapping sub-clips, and then compute the rFVD score on totally 80000 video clips of length 16.

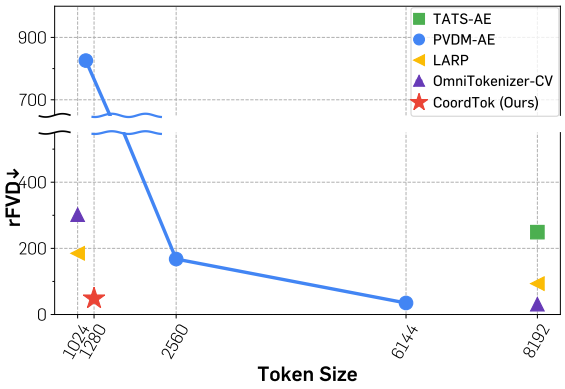


Figure 10. rFVD scores of video tokenizers, evaluated on 16-frame videos, with respect to the token size used for encoding 128-frame videos. ↓ indicates lower values are better.

F. Additional Qualitative Results

In Figure 11, we provide additional video reconstruction results from CoordTok. In addition, in Figures 12 and 13, we provide unconditional video generation results from CoordTok-SiT-L/2.

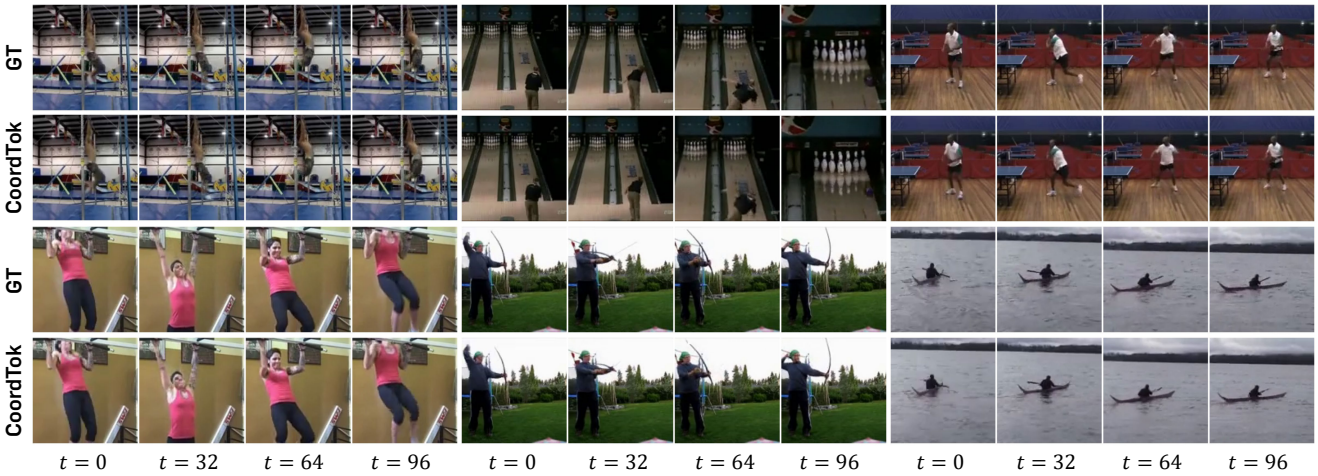


Figure 11. Additional 128-frame, 128×128 resolution video reconstruction results from CoordTok (Ours) trained on the UCF-101 dataset [42]. For each frame, we visualize the ground-truth (GT) and reconstructed pixels from CoordTok.

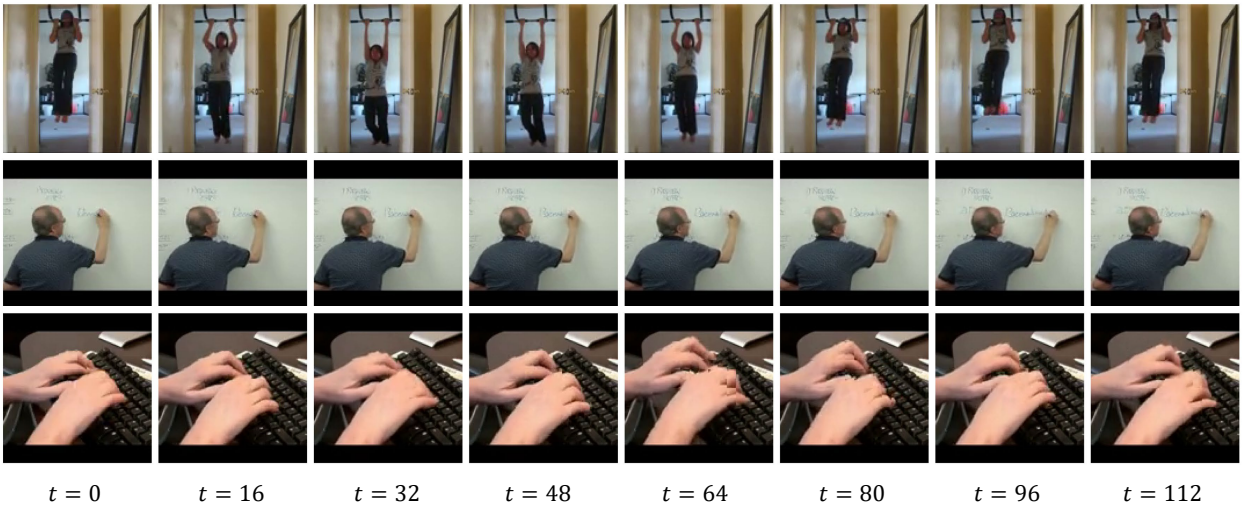


Figure 12. Unconditional 128-frame, 128×128 resolution video generation results from CoordTok-SiT-L/2 trained on 128-frame videos from the UCF-101 dataset [42].

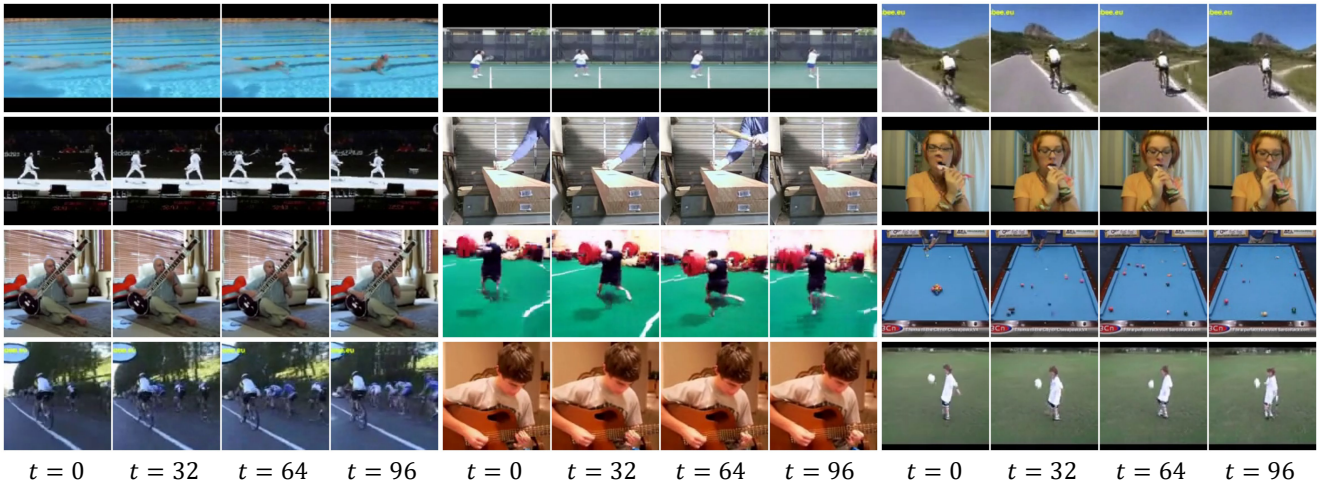


Figure 13. Unconditional 128-frame, 128×128 resolution video generation results from CoordTok-SiT-L/2 trained on 128-frame videos from the UCF-101 dataset [42].